

Protocol

Link to Github repository(code to the current task is in the branch micro_basics):
https://github.com/hhafiya/software_architecture.git

Overview

Task 1 implements a simplified model of a banking system based on a microservices architecture. The main goal is to create a foundation for a distributed system where each component is responsible for a specific task, communicating via the HTTP(following the REST API).

The system consists of three microservices, each running in a separate Docker container and managed via Docker Compose:

1. Facade-service

- Acts as the single entry point for clients
- Generates a unique transaction_ID
- Forwards data to both the logging and counter services.
- Measures the response time of internal service calls to compare implementation performance.

2. Logging-service

- Stores all received transaction messages in memory using the transaction_ID as the key.

3. Counter-service:

- Stores current balances in hash table with the user_id as the key and the balance as the value.

Functional Correctness Verification(with OpenAPI docs)

Scenario 1: Multiple Transactions for Different Clients

{user_1, 100}

Curl

```
curl -X 'POST' \
  'http://localhost:8080/transaction' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_1",
    "amount": 100
  }'
```

Response body

```
{
  "transaction_id": "1771422625103",
  "balance": 100
}
```

```
counter-service-1 | INFO: 172.18.0.4:34190 - "POST /update HTTP/1.1" 200 OK
logging-service-1 | LOG: Received transaction 1771422625103
logging-service-1 | INFO: 172.18.0.4:60108 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:63194 - "POST /transaction HTTP/1.1" 200 OK
```

{user_2, 200}

Curl

```
curl -X 'POST' \
  'http://localhost:8080/transaction' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_2",
    "amount": 200
  }'
```

Response body

```
{
  "transaction_id": "1771422745609",
  "balance": 200
}
```

```
counter-service-1 | INFO: 172.18.0.4:38576 - "POST /update HTTP/1.1" 200 OK
logging-service-1 | LOG: Received transaction 1771422745609
logging-service-1 | INFO: 172.18.0.4:53388 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:55192 - "POST /transaction HTTP/1.1" 200 OK
```

{user_3, 300}

Curl

```
curl -X 'POST' \
  'http://localhost:8080/transaction' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_3",
    "amount": 300
  }'
```

Response body

```
{
  "transaction_id": "1771422858533",
  "balance": 300
}
```

```
logging-service-1 | LOG: Received transaction 1771422858533
logging-service-1 | INFO: 172.18.0.4:44422 - "POST /log HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.18.0.4:57786 - "POST /update HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:55208 - "POST /transaction HTTP/1.1" 200 OK
```

GET /accounts

```
counter-service-1 | INFO: 172.18.0.4:38010 - "GET /balances HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:62782 - "GET /accounts HTTP/1.1" 200 OK
```

Curl

```
curl -X 'GET' \
  'http://localhost:8080/accounts' \
  -H 'accept: application/json'
```

Response body

```
{
  "user_1": 100,
  "user_2": 200,
  "user_3": 300
}
```

GET /user/user_id

Curl

```
curl -X 'GET' \
  'http://localhost:8080/user/user_1' \
  -H 'accept: application/json'
```

Response body

```
{
  "balance": 100,
  "transactions": [
    {
      "transaction_id": "1771422625103",
      "user_id": "user_1",
      "amount": 100
    }
  ]
}
```

```
counter-service-1 | INFO: 172.18.0.4:46614 - "GET /balance/user_1 HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.18.0.4:59984 - "GET /logs/user_1 HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:63886 - "GET /user/user_1 HTTP/1.1" 200 OK
```

Curl

```
curl -X 'GET' \
  'http://localhost:8080/user/user_2' \
  -H 'accept: application/json'
```

Response body

```
{
  "balance": 200,
  "transactions": [
    {
      "transaction_id": "1771422745609",
      "user_id": "user_2",
      "amount": 200
    }
  ]
}
```

```
counter-service-1 | INFO: 172.18.0.4:39986 - "GET /balance/user_2 HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.18.0.4:44576 - "GET /logs/user_2 HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:62728 - "GET /user/user_2 HTTP/1.1" 200 OK
```

Curl

```
curl -X 'GET' \
  'http://localhost:8080/user/user_3' \
  -H 'accept: application/json'
```

Response body

```
{
  "balance": 300,
  "transactions": [
    {
      "transaction_id": "1771422858533",
      "user_id": "user_3",
      "amount": 300
    }
  ]
}
```

```
logging-service-1 | INFO: 172.18.0.4:48886 - "GET /logs/user_3 HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.18.0.4:48844 - "GET /balance/user_3 HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:55358 - "GET /user/user_3 HTTP/1.1" 200 OK
```

Scenario 2: Multiple Transactions for One Client

Curl

```
curl -X 'POST' \
  'http://localhost:8080/transaction' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_1",
    "amount": 1000
  }'
```

Response body

```
{
  "transaction_id": "1771423504161",
  "balance": 1100
}
```

```
counter-service-1 | INFO: 172.18.0.4:53346 - "POST /update HTTP/1.1" 200 OK
logging-service-1 | LOG: Received transaction 1771423504161
logging-service-1 | INFO: 172.18.0.4:50166 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:57766 - "POST /transaction HTTP/1.1" 200 OK
```

Curl

```
curl -X 'POST' \
  'http://localhost:8080/transaction' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_1",
    "amount": 10
  }'
```

Response body

```
{
  "transaction_id": "1771423614765",
  "balance": 1110
}
```

```
counter-service-1 | INFO: 172.18.0.4:37792 - "POST /update HTTP/1.1" 200 OK
logging-service-1 | LOG: Received transaction 1771423614765
logging-service-1 | INFO: 172.18.0.4:48768 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:57584 - "POST /transaction HTTP/1.1" 200 OK
```

Curl

```
curl -X 'POST' \
  'http://localhost:8080/transaction' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_1",
    "amount": -2000
  }'
```

Response body

```
{
  "transaction_id": "1771423693578",
  "balance": -890
}
```

```
logging-service-1 | LOG: Received transaction 1771423693578
logging-service-1 | INFO: 172.18.0.4:34464 - "POST /log HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.18.0.4:36904 - "POST /update HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:63874 - "POST /transaction HTTP/1.1" 200 OK
```

GET /accounts

From the response you can observe that result balances were counted correctly.

Curl

```
curl -X 'GET' \
'http://localhost:8080/accounts' \
-H 'accept: application/json'
```

Response body

```
{
  "user_1": -890,
  "user_2": 200,
  "user_3": 300
}
```

```
counter-service-1 | INFO: 172.18.0.4:54224 - "GET /balances HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:60892 - "GET /accounts HTTP/1.1" 200 OK
```

GET /user/user_id

And user_1 has all three transactions I made earlier stored as expected.

Curl

```
curl -X 'GET' \
'http://localhost:8080/user/user_1' \
-H 'accept: application/json'
```

Response body

```
{
  "balance": -890,
  "transactions": [
    {
      "transaction_id": "1771422625103",
      "user_id": "user_1",
      "amount": 100
    },
    {
      "transaction_id": "1771423504161",
      "user_id": "user_1",
      "amount": 1000
    },
    {
      "transaction_id": "1771423614765",
      "user_id": "user_1",
      "amount": 10
    },
    {
      "transaction_id": "1771423693578",
      "user_id": "user_1",
      "amount": -2000
    }
  ]
}
```

```
counter-service-1 | INFO: 172.18.0.4:57128 - "GET /balance/user_1 HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.18.0.4:44090 - "GET /logs/user_1 HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:65082 - "GET /user/user_1 HTTP/1.1" 200 OK
```

The core functions have been proven to work correctly, and system logs show that all requests were successful. The same applies to the logic of the facade service when requesting the logging and counter services.

Performance Testing and Analysis

To evaluate the system I implemented an HTTP client for two cases:

1. 10 concurrent clients perform 10000 transactions each. Each transaction adds +1 to a unique user account. The total number of requests must be 100000 and the final balance on each of the 10 accounts must be exactly 10000.
2. 10 concurrent clients perform 10000 transactions each. All 10 send transactions to the same user_id, adding +1 in each request. The total number of requests must be 100000 and the final balance on the shared account must be exactly 100000.

Result:

```
Resetting system...
10 clients 10 accounts scenario
Total time: 215.4941074170638
Requests/sec: 464.04981184224044

Service contribution:
Logging total time: 1272.5887620283738
Counter total time: 1255.0846905879062
Request count: 100000

Final balances:
{'user_0': 10000, 'user_1': 10000, 'user_3': 10000, 'user_5': 10000, 'user_4': 10000, 'user_2': 10000, 'user_7': 10000, 'user_6': 10000, 'user_8': 10000, 'user_9': 10000}
```

Case 1

```
Resetting system...
10 clients 1 account scenario
Total time: 232.74651308299508
Requests/sec: 429.6519792085607

Service contribution:
Logging total time: 1371.2304886489856
Counter total time: 1358.6068351417089
Request count: 100000

Final balances:
{'shared_user': 100000}
```

Case 2

Service Contribution Analysis

In both cases, the cumulative time for the logging-service and counter-service is significantly higher the total execution time. For case 1: ~1272s and ~1255 s vs ~215s, for case 2: ~1358s and ~1371s vs 232s. That means that the system is concurrent, requests are processed in parallel, so the total workload is finished faster than the sum of individual operations.

For case 1 and case 2, the logging service has a higher response time compared to counter service. The first reason is synchronous I/O operation(console logging via the print) in logging service. The second reason is that the logging service manages a growing set of unique keys and stores complex objects(full transaction history), while counter service performs a lightweight integer addition and stores it in the smaller set of keys.

Case 1 vs Case 2

Throughput in Scenario 2(~429) decreased by approximately 7.4% compared to Scenario 1(~464), while the total time of case 1 was lower, around 215s and of case 2 it was ~232s. This is because of high resource contention. When all the clients target the same key in balances table, the overhead for memory synchronization increases.

Correctness

For both cases final balances were counted as excepted. Case 1: 10 users with 10000 on each account, case 2: 1 account with 100000. The total number of transactions for both were exactly 100000.